Table 1: End-to-end execution time (s, median per iteration) and speedup split by pass type. ADT fields = non-recursive fields annotated with `@BENCH adt_fields`; SoA bufs = 1+ non-recursive field slots across all constructors (recursive children are stored in the tag buffer). Speedup >1× means SoA is faster; **bold** marks >1.1×.

Program	**ADT**	**SoA**	**Fold passes**			**Map passes**		
	fields	bufs	AoS (s)	SoA (s)	Speedup	AoS (s)	SoA (s)	Speedup
Compiler	9	8	0.1328	0.0568	**2.34**×	0.1624	0.1856	0.88×
DBQuery	15	13	0.0633	0.0374	**1.69**×	0.1743	0.2488	0.70×
DecisionTree	7	6	4.175	3.293	**1.27**×	–	–	–
DomTree	15	14	0.1465	0.0629	**2.33**×	0.0588	0.0881	0.67×
KDTree	19	12	0.3742	0.2939	**1.27**×	–	–	–
LinearListReduction	11	11	0.0315	0.0137	**2.30**×	–	–	–
List	2	2	0.2767	0.2645	1.05×	0.3027	0.3394	0.89×
MonoTree	3	2	0.0235	0.0225	1.05×	0.0470	0.0431	1.09×
ObjectGraph	5	4	0.0068	0.0053	**1.27**×	0.0185	0.0202	0.92×
OctTree	16	9	0.0156	0.0131	**1.19**×	0.0385	0.0492	0.78×
PiecewiseFunctions	8	7	0.0105	0.0054	**1.95**×	0.0449	0.0508	0.88×
TernaryTree	5	3	0.0304	0.0313	0.97×	0.0840	0.0786	1.07×
Trie	10	9	0.0259	0.0112	**2.31**×	0.0751	0.0945	0.79×

Table 2: PAPI speedup summary across all passes. For each program and counter, speedup is AoS/SoA using summed per-pass median counters. >1× means SoA has fewer events.

Program	**CYC**	**INS**	**L1D**	**L1I**	**L2D**	**L2I**	**LLC**
Compiler	**1.36**×	0.48×	**3.11**×	0.81×	**12.01**×	0.44×	**7.36**×
DBQuery	0.75×	0.37×	**2.16**×	0.60×	**1.40**×	0.11×	**6.01**×
DecisionTree	**1.27**×	0.69×	**20.60**×	**1.78**×	0.90×	**1.91**×	**36.18**×
DomTree	**1.47**×	0.53×	**2.79**×	0.52×	**19.97**×	0.43×	**10.18**×
KDTree	**1.27**×	0.87×	**5.72**×	**2.01**×	0.28×	**2.03**×	**3.38**×
LinearListReduction	**2.32**×	0.90×	**14.29**×	**4.52**×	**57.89**×	**4.70**×	**16.77**×
List	0.97×	0.74×	**3.46**×	0.62×	0.13×	0.56×	**1.53**×
MonoTree	**1.12**×	0.82×	0.40×	0.84×	0.25×	0.92×	**1.11**×
ObjectGraph	0.99×	0.69×	**4.39**×	1.08×	0.67×	**1.27**×	**4.87**×
OctTree	0.85×	0.55×	0.29×	0.36×	0.28×	0.60×	**5.94**×
PiecewiseFunctions	0.97×	0.52×	**2.22**×	**1.16**×	**1.29**×	0.73×	**6.06**×
TernaryTree	1.08×	0.78×	0.25×	0.50×	0.25×	0.92×	**1.44**×
Trie	0.93×	0.47×	**3.12**×	0.60×	**2.59**×	0.14×	**8.67**×
Geomean	**1.13**×	0.62×	**2.42**×	0.91×	**1.28**×	0.72×	**5.41**×

Table 3: PAPI counter totals across all passes (AoS/SoA). Each entry is the sum of per-pass median counter values.

Program	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
Compiler	9.8e+08/**7.2e+08**	**9.9e+08**/2.1e+09	1.1e+07/**3.7e+06**	**2.4e+05**/3.0e+05	9.8e+05/**8.1e+04**	**2.0e+04**/4.6e+04	5.1e+07/**7.0e+06**
DBQuery	**6.9e+08**/9.2e+08	**1.3e+09**/3.5e+09	1.0e+07/**4.8e+06**	**2.1e+05**/3.5e+05	1.1e+05/**7.7e+04**	**1.1e+04**/1.0e+05	2.6e+07/**4.3e+06**
DecisionTree	2.1e+10/**1.7e+10**	**4.3e+10**/6.3e+10	5.9e+08/**2.9e+07**	1.9e+05/**1.1e+05**	**1.9e+06**/2.1e+06	1.9e+05/**1.0e+05**	1.1e+09/**3.0e+07**
DomTree	8.7e+08/**5.9e+08**	**1.2e+09**/2.3e+09	1.2e+07/**4.2e+06**	**1.1e+05**/2.1e+05	1.0e+06/**5.1e+04**	**1.6e+04**/3.6e+04	5.6e+07/**5.5e+06**
KDTree	1.9e+09/**1.5e+09**	**6.9e+09**/8.0e+09	2.1e+07/**3.7e+06**	1.9e+03/**9.3e+02**	**3.8e+04**/1.4e+05	1.8e+03/**9.1e+02**	8.2e+07/**2.4e+07**
LinearListReduction	1.6e+08/**6.9e+07**	**9.0e+07**/1.0e+08	1.7e+06/**1.2e+05**	1.2e+03/**2.6e+02**	2.7e+05/**4.6e+03**	1.2e+03/**2.5e+02**	1.2e+07/**7.3e+05**
List	**2.2e+09**/2.3e+09	**4.5e+09**/6.1e+09	1.2e+07/**3.5e+06**	**3.4e+05**/5.4e+05	**2.3e+04**/1.8e+05	**1.3e+04**/2.3e+04	3.9e+07/**2.5e+07**
MonoTree	2.9e+08/**2.6e+08**	**9.1e+08**/1.1e+09	**1.9e+05**/4.8e+05	**6.7e+04**/8.0e+04	**3.6e+03**/1.5e+04	**5.1e+03**/5.6e+03	1.9e+06/**1.7e+06**
ObjectGraph	**8.4e+07**/8.5e+07	**2.6e+08**/3.8e+08	5.0e+05/**1.1e+05**	3.8e+04/**3.5e+04**	**3.3e+03**/5.0e+03	4.0e+03/**3.2e+03**	5.7e+05/**1.2e+05**
OctTree	**1.7e+08**/2.0e+08	**4.1e+08**/7.5e+08	**3.1e+05**/1.1e+06	**5.0e+04**/1.4e+05	**4.7e+03**/1.7e+04	**3.4e+03**/5.7e+03	3.5e+06/**6.0e+05**
PiecewiseFunctions	**1.6e+08**/1.6e+08	**3.5e+08**/6.6e+08	1.9e+06/**8.4e+05**	9.0e+04/**7.8e+04**	1.6e+04/**1.2e+04**	**8.3e+03**/1.1e+04	3.3e+06/**5.4e+05**
TernaryTree	4.2e+08/**3.9e+08**	**1.2e+09**/1.6e+09	**3.4e+05**/1.4e+06	**7.4e+04**/1.5e+05	**5.1e+03**/2.0e+04	**4.0e+03**/4.3e+03	3.3e+06/**2.3e+06**
Trie	**3.0e+08**/3.2e+08	**5.2e+08**/1.1e+09	4.0e+06/**1.3e+06**	**9.8e+04**/1.6e+05	6.4e+04/**2.5e+04**	**4.4e+03**/3.2e+04	1.0e+07/**1.2e+06**

Table 4: Mutable vs immutable cursor comparison (times only). Times are median per iteration (s). **Bold** marks the fastest time across all four variants.

Program	**Am**	**Ai**	**Sm**	**Si**
Compiler	0.2952	1.205	**0.2424**	*OOM*
DBQuery	**0.2376**	0.3299	0.2862	1.374
DecisionTree	4.175	5.420	**3.293**	28.46
DomTree	0.2053	0.3529	**0.1510**	1.027
KDTree	0.3742	0.4441	**0.2939**	2.627
LinearListReduction	0.0315	0.1630	**0.0137**	*OOM*
List	**0.5794**	*OOM*	0.6039	*OOM*
MonoTree	0.0705	0.1505	**0.0656**	0.2949
ObjectGraph	**0.0253**	0.0452	0.0255	0.0780
OctTree	**0.0541**	0.0610	0.0624	0.2496
PiecewiseFunctions	**0.0555**	0.0757	0.0562	0.3007
TernaryTree	0.1144	0.1502	**0.1099**	0.6167
Trie	**0.1010**	0.1330	0.1057	0.4751

Table 5: Mutable vs immutable cursor comparison (speedups). Shown: AoS-mut/SoA-mut, AoS-imm/AoS-mut, AoS-imm/SoA-mut, AoS-imm/SoA-imm. >1× means the denominator is faster.

Program	**Am/Sm**	**Ai/Am**	**Ai/Sm**	**Ai/Si**
Compiler	**1.22**×	**4.08**×	**4.97**×	–
DBQuery	0.83×	**1.39**×	**1.15**×	0.24×
DecisionTree	**1.27**×	**1.30**×	**1.65**×	0.19×
DomTree	**1.36**×	**1.72**×	**2.34**×	0.34×
KDTree	**1.27**×	**1.19**×	**1.51**×	0.17×
LinearListReduction	**2.30**×	**5.17**×	**11.89**×	–
List	0.96×	–	–	–
MonoTree	1.08×	**2.13**×	**2.29**×	0.51×
ObjectGraph	0.99×	**1.79**×	**1.77**×	0.58×
OctTree	0.87×	**1.13**×	0.98×	0.24×
PiecewiseFunctions	0.99×	**1.36**×	**1.35**×	0.25×
TernaryTree	1.04×	**1.31**×	**1.37**×	0.24×
Trie	0.96×	**1.32**×	**1.26**×	0.28×

Table 6: Per-pass performance for `Compiler`, ADT has 9 fields, SoA uses 8 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Si**	**Am/Sm**	**Ai/Am**	**Ai/Sm**	**Ai/Si**
blockCountPass	F	2/9	78%	0.0129±0.00009	0.0838±0.00002	**0.0019±0.00000**	*OOM*	**6.71×**	**6.50×**	**43.63×**	–
branchStatsPass	F	2/9	78%	0.0122±0.00001	0.0985±0.00131	**0.0080±0.00000**	*OOM*	**1.53×**	**8.07×**	**12.37×**	–
castInstCountPass	F	3/9	67%	0.0189±0.00001	0.0974±0.00003	**0.0013±0.00000**	*OOM*	**14.63×**	**5.16×**	**75.56×**	–
has cycle	F	4/9	56%	0.0255±0.00017	0.0995±0.00113	**0.0173±0.00016**	*OOM*	**1.47×**	**3.90×**	**5.75×**	–
instCountPass	F	2/9	78%	0.0129±0.00038	0.0888±0.00004	**0.0019±0.00000**	*OOM*	**6.59×**	**6.91×**	**45.52×**	–
latencyModelPass	F	3/9	67%	0.0124±0.00001	0.0976±0.00004	**0.0075±0.00001**	*OOM*	**1.66×**	**7.87×**	**13.07×**	–
memoryOpStatsPass	F	3/9	67%	0.0126±0.00002	0.0974±0.00083	**0.0080±0.00000**	*OOM*	**1.57×**	**7.76×**	**12.17×**	–
stripSideEffectsPass	M	7/9	22%	**0.0814±0.00019**	0.1885±0.00020	0.0926±0.00015	*OOM*	0.88×	**2.31×**	**2.04×**	–
targetReturnPass	M	9/9	0%	**0.0810±0.00161**	0.1882±0.00029	0.0930±0.00024	*OOM*	0.87×	**2.32×**	**2.02×**	–
throughputModelPass	F	3/9	67%	0.0124±0.00001	0.0976±0.00137	**0.0075±0.00000**	*OOM*	**1.66×**	**7.88×**	**13.06×**	–
verifyIR	F	9/9	0%	0.0131±0.00006	0.0682±0.00184	**0.0035±0.00006**	*OOM*	**3.78×**	**5.21×**	**19.68×**	–
Total				0.2952	1.205	**0.2424**	–	**1.22×**	**4.08×**	**4.97×**	–
Geomean								**2.44×**	**5.33×**	**13.00×**	–

Table 7: Per-pass PAPI counters for `Compiler` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
blockCountPass	F	6.4e+07/**9.7e+06**	5.1e+07/**5.1e+07**	1.3e+06/**2.0e+04**	7.6e+02/**2.3e+02**	2.5e+05/**1.2e+03**	7.4e+02/**2.2e+02**	4.7e+06/**1.1e+03**
branchStatsPass	F	6.2e+07/**4.0e+07**	**7.9e+07**/8.6e+07	6.8e+05/**1.0e+05**	6.9e+02/**2.4e+02**	3.1e+04/**3.5e+03**	6.7e+02/**2.4e+02**	4.7e+06/**3.4e+05**
castInstCountPass	F	9.5e+07/**6.5e+06**	5.1e+07/**3.2e+07**	1.3e+04/**1.1e+04**	2.4e+02/**2.3e+02**	**9.2e+02**/1.2e+03	**2.2e+02**/2.3e+02	4.7e+06/**6.0e+02**
has cycle	F	1.3e+08/**8.8e+07**	**1.1e+08**/1.5e+08	2.5e+06/**3.2e+05**	7.8e+02/**3.2e+02**	1.8e+05/**2.2e+04**	7.6e+02/**3.1e+02**	5.6e+06/**2.4e+06**
instCountPass	F	6.4e+07/**9.9e+06**	**5.6e+07**/5.7e+07	1.5e+06/**1.5e+04**	9.0e+02/**2.4e+02**	2.6e+05/**1.2e+03**	8.8e+02/**2.3e+02**	4.7e+06/**4.1e+03**
latencyModelPass	F	6.3e+07/**3.8e+07**	**6.2e+07**/6.9e+07	7.6e+05/**1.4e+05**	7.1e+02/**2.3e+02**	3.8e+04/**3.6e+03**	6.9e+02/**2.2e+02**	4.8e+06/**3.7e+05**
memoryOpStatsPass	F	6.3e+07/**4.0e+07**	**8.5e+07**/9.1e+07	5.2e+05/**9.3e+04**	9.8e+02/**2.3e+02**	2.4e+04/**3.2e+03**	9.7e+02/**2.3e+02**	4.7e+06/**3.5e+05**
stripSideEffectsPass	M	**1.6e+08**/2.2e+08	**1.8e+08**/6.5e+08	**8.5e+05**/1.3e+06	**1.2e+05**/1.5e+05	**7.6e+03**/1.7e+04	**4.9e+03**/1.6e+04	4.0e+06/**1.2e+06**
targetReturnPass	M	**1.5e+08**/2.2e+08	**1.7e+08**/7.1e+08	1.6e+06/**1.5e+06**	**1.2e+05**/1.5e+05	**7.2e+03**/2.2e+04	**8.5e+03**/2.8e+04	4.1e+06/**1.5e+06**
throughputModelPass	F	6.3e+07/**3.8e+07**	**6.2e+07**/6.9e+07	7.6e+05/**1.6e+05**	7.4e+02/**2.3e+02**	3.6e+04/**3.0e+03**	7.3e+02/**2.2e+02**	4.7e+06/**3.8e+05**
verifyIR	F	6.6e+07/**1.7e+07**	**7.4e+07**/8.6e+07	9.4e+05/**1.7e+04**	1.4e+03/**3.1e+02**	1.5e+05/**3.9e+03**	1.4e+03/**2.9e+02**	4.6e+06/**4.6e+05**
Total		9.8e+08/7.2e+08	9.9e+08/2.1e+09	1.1e+07/3.7e+06	2.4e+05/3.0e+05	9.8e+05/8.1e+04	2.0e+04/4.6e+04	5.1e+07/7.0e+06

Table 8: Per-pass PAPI counters for `Compiler` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Ai/Si)**	**INS (Ai/Si)**	**L1D (Ai/Si)**	**L1I (Ai/Si)**	**L2D (Ai/Si)**	**L2I (Ai/Si)**	**LLC (Ai/Si)**
blockCountPass	F	4.2e+08/–	2.2e+08/–	2.0e+06/–	1.2e+03/–	5.1e+04/–	1.1e+03/–	1.5e+07/–
branchStatsPass	F	5.0e+08/–	2.5e+08/–	2.7e+06/–	1.3e+03/–	8.8e+04/–	1.3e+03/–	1.7e+07/–
castInstCountPass	F	4.9e+08/–	2.5e+08/–	2.7e+06/–	6.4e+02/–	9.0e+04/–	6.2e+02/–	1.7e+07/–
has cycle	F	5.0e+08/–	2.6e+08/–	2.7e+06/–	7.4e+02/–	8.4e+04/–	7.2e+02/–	1.7e+07/–
instCountPass	F	4.4e+08/–	2.2e+08/–	2.0e+06/–	1.2e+03/–	5.1e+04/–	1.2e+03/–	1.4e+07/–
latencyModelPass	F	4.9e+08/–	2.4e+08/–	2.7e+06/–	7.7e+02/–	8.7e+04/–	7.5e+02/–	1.7e+07/–
memoryOpStatsPass	F	4.9e+08/–	2.6e+08/–	2.7e+06/–	5.8e+02/–	9.0e+04/–	5.7e+02/–	1.7e+07/–
stripSideEffectsPass	M	6.9e+08/–	4.2e+08/–	1.6e+06/–	1.4e+05/–	1.7e+05/–	2.3e+04/–	2.4e+07/–
targetReturnPass	M	6.9e+08/–	4.0e+08/–	1.8e+06/–	1.8e+05/–	1.6e+05/–	3.0e+04/–	2.4e+07/–
throughputModelPass	F	4.9e+08/–	2.4e+08/–	2.7e+06/–	4.9e+02/–	8.7e+04/–	4.8e+02/–	1.7e+07/–
verifyIR	F	3.4e+08/–	1.4e+08/–	2.1e+06/–	1.8e+03/–	1.8e+05/–	1.8e+03/–	1.2e+07/–
Total		5.6e+09/–	2.9e+09/–	2.6e+07/–	3.3e+05/–	1.1e+06/–	6.2e+04/–	1.9e+08/–

Table 9: Per-pass performance for `DBQuery`, ADT has 15 fields, SoA uses 13 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	**T**	**Uses**	**Dead%**	**Am**	**Ai**	**Sm**	**Si**	**Am/Sm**	**Ai/Am**	**Ai/Sm**	**Ai/Si**
clearQueryFlags	M	14/15	7%	**0.0874±0.00024**	0.0980±0.00012	0.1230±0.00020	0.3785±0.00191	0.71×	**1.12×**	0.80×	0.26×
countJoins	F	3/15	80%	0.0156±0.00001	0.0324±0.00001	**0.0070±0.00001**	0.1390±0.00114	**2.24×**	**2.08×**	**4.65×**	0.23×
scaleCosts	M	15/15	0%	**0.0870±0.00016**	0.0972±0.00022	0.1258±0.00013	0.3850±0.00119	0.69×	**1.12×**	0.77×	0.25×
sumCost	F	6/15	60%	0.0158±0.00068	0.0340±0.00020	**0.0098±0.00006**	0.1580±0.00003	**1.61×**	**2.15×**	**3.47×**	0.22×
sumMemory	F	6/15	60%	0.0159±0.00001	0.0342±0.00001	**0.0103±0.00094**	0.1568±0.00005	**1.55×**	**2.15×**	**3.33×**	0.22×
sumRows	F	6/15	60%	0.0160±0.00001	0.0341±0.00001	**0.0104±0.00004**	0.1571±0.00119	**1.54×**	**2.14×**	**3.29×**	0.22×
Total				**0.2376**	0.3299	0.2862	1.374	0.83×	**1.39×**	**1.15×**	0.24×
Geomean								**1.27×**	**1.72×**	**2.19×**	0.23×

Table 10: Per-pass PAPI counters for `DBQuery` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
clearQueryFlags	M	**1.9e+08**/3.6e+08	**3.2e+08**/1.3e+09	**8.1e+05**/2.2e+06	**1.0e+05**/1.7e+05	**1.1e+04**/3.2e+04	**5.2e+03**/4.5e+04	3.6e+06/**1.4e+06**
countJoins	F	7.8e+07/**3.5e+07**	1.3e+08/**1.2e+08**	2.3e+06/**4.3e+04**	**3.0e+02**/3.2e+02	2.3e+04/**1.6e+03**	**2.8e+02**/3.1e+02	4.5e+06/**4.6e+03**
scaleCosts	M	**1.9e+08**/3.7e+08	**3.8e+08**/1.4e+09	**1.2e+06**/2.2e+06	**1.1e+05**/1.8e+05	**1.2e+04**/3.0e+04	**4.6e+03**/5.4e+04	3.7e+06/**1.4e+06**
sumCost	F	8.0e+07/**4.9e+07**	**1.5e+08**/2.1e+08	2.1e+06/**1.6e+05**	3.6e+02/**2.9e+02**	2.1e+04/**4.4e+03**	3.5e+02/**2.8e+02**	4.7e+06/**5.1e+05**
sumMemory	F	8.0e+07/**5.1e+07**	**1.5e+08**/2.1e+08	2.0e+06/**1.4e+05**	3.3e+02/**3.0e+02**	2.1e+04/**4.4e+03**	3.2e+02/**2.8e+02**	4.7e+06/**4.9e+05**
sumRows	F	8.0e+07/**5.2e+07**	**1.5e+08**/2.1e+08	2.0e+06/**1.1e+05**	4.1e+02/**2.6e+02**	2.0e+04/**4.4e+03**	3.8e+02/**2.5e+02**	4.7e+06/**5.0e+05**
Total		6.9e+08/9.2e+08	1.3e+09/3.5e+09	1.0e+07/4.8e+06	2.1e+05/3.5e+05	1.1e+05/7.7e+04	1.1e+04/1.0e+05	2.6e+07/4.3e+06

Table 11: Per-pass PAPI counters for `DBQuery` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Ai/Si)**	**INS (Ai/Si)**	**L1D (Ai/Si)**	**L1I (Ai/Si)**	**L2D (Ai/Si)**	**L2I (Ai/Si)**	**LLC (Ai/Si)**
clearQueryFlags	M	**2.5e+08**/1.6e+09	**5.6e+08**/3.7e+09	**5.7e+05**/9.9e+06	**1.3e+05**/5.7e+05	**5.1e+03**/4.8e+04	**6.0e+03**/1.0e+05	3.4e+06/**1.6e+06**
countJoins	F	**1.6e+08**/7.1e+08	**3.5e+08**/1.2e+09	2.9e+06/**1.7e+05**	**3.3e+02**/2.3e+03	**4.9e+03**/1.2e+04	**3.2e+02**/3.8e+02	3.1e+06/**2.6e+04**
scaleCosts	M	**2.4e+08**/1.7e+09	**6.0e+08**/3.7e+09	**5.9e+05**/1.1e+07	**1.5e+05**/1.0e+06	**6.0e+03**/5.8e+04	**5.2e+03**/8.0e+04	3.3e+06/**1.4e+06**
sumCost	F	**1.7e+08**/8.0e+08	**3.8e+08**/1.2e+09	**6.4e+05**/1.2e+06	**2.8e+02**/3.6e+03	**1.9e+03**/2.5e+04	**2.7e+02**/1.5e+03	3.3e+06/**4.1e+05**
sumMemory	F	**1.7e+08**/7.9e+08	**3.8e+08**/1.2e+09	**6.5e+05**/1.3e+06	**3.5e+02**/3.2e+03	**1.6e+03**/2.7e+04	**3.4e+02**/1.7e+03	3.3e+06/**4.0e+05**
sumRows	F	**1.7e+08**/8.0e+08	**3.8e+08**/1.2e+09	**5.8e+05**/1.3e+06	**3.0e+02**/1.9e+03	**1.7e+03**/2.7e+04	**2.9e+02**/8.8e+02	3.3e+06/**4.1e+05**
Total		1.2e+09/6.4e+09	2.6e+09/1.2e+10	5.9e+06/2.5e+07	2.8e+05/1.6e+06	2.1e+04/2.0e+05	1.2e+04/1.8e+05	2.0e+07/4.3e+06

Table 12: Per-pass performance for DecisionTree, ADT has 7 fields, SoA uses 6 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
classify Batch	F	5/7	29%	3.571±0.0051	4.019±0.0044	**2.920±0.0045**	24.40±0.013	**1.22×**	**1.13×**	**1.38×**	0.16×
classify Depth	F	4/7	43%	0.0511±0.00064	0.0573±0.00138	**0.0398±0.00002**	0.3480±0.00179	**1.28×**	**1.12×**	**1.44×**	0.16×
classify tree	F	5/7	29%	0.0513±0.00001	0.0573±0.00001	**0.0399±0.00002**	0.3471±0.00204	**1.29×**	**1.12×**	**1.44×**	0.17×
countFeatureUses	F	3/7	57%	0.0560±0.00001	0.1418±0.00127	**0.0321±0.00003**	0.3941±0.00185	**1.74×**	**2.53×**	**4.41×**	0.36×
countLeaves	F	2/7	71%	0.0558±0.00003	0.1446±0.00128	**0.0317±0.00001**	0.3741±0.00168	**1.76×**	**2.59×**	**4.57×**	0.39×
countNodes	F	2/7	71%	0.0556±0.00002	0.1424±0.00098	**0.0317±0.00002**	0.3755±0.00159	**1.75×**	**2.56×**	**4.49×**	0.38×
countSmallLeaves	F	3/7	57%	0.0564±0.00084	0.1417±0.00103	**0.0327±0.00003**	0.3234±0.00009	**1.72×**	**2.51×**	**4.33×**	0.44×
inferenceCost	F	2/7	71%	0.0554±0.00002	0.1454±0.00131	**0.0340±0.00001**	0.3790±0.00163	**1.63×**	**2.62×**	**4.28×**	0.38×
sumImpurity	F	3/7	57%	0.0551±0.00001	0.1406±0.00017	**0.0318±0.00003**	0.3859±0.00253	**1.73×**	**2.55×**	**4.42×**	0.36×
sumPathLengths	F	3/7	57%	0.0571±0.00002	0.1458±0.00002	**0.0336±0.00002**	0.3783±0.00171	**1.70×**	**2.55×**	**4.33×**	0.39×
sumSamples	F	3/7	57%	0.0549±0.00001	0.1395±0.00126	**0.0327±0.00002**	0.3798±0.00167	**1.68×**	**2.54×**	**4.27×**	0.37×
treeDepth	F	2/7	71%	0.0556±0.00093	0.1442±0.00005	**0.0332±0.00001**	0.3774±0.00120	**1.68×**	**2.59×**	**4.34×**	0.38×
Total				4.175	5.420	**3.293**	28.46	**1.27×**	**1.30×**	**1.65×**	0.19×
Geomean								**1.59×**	**2.08×**	**3.30×**	0.31×

Table 13: Per-pass PAPI counters for DecisionTree (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
classify Batch	F	1.8e+10/**1.5e+10**	**3.6e+10**/5.5e+10	5.1e+08/**2.6e+07**	1.6e+05/**9.8e+04**	**1.6e+06**/2.0e+06	1.5e+05/**9.1e+04**	9.2e+08/**1.8e+07**
classify Depth	F	2.6e+08/**2.0e+08**	**5.2e+08**/7.8e+08	7.3e+06/**3.9e+05**	2.9e+03/**1.2e+03**	**2.5e+04**/2.8e+04	2.9e+03/**1.1e+03**	1.3e+07/**2.9e+05**
classify tree	F	2.6e+08/**2.0e+08**	**5.1e+08**/7.8e+08	7.3e+06/**3.9e+05**	3.1e+03/**1.2e+03**	**2.2e+04**/2.8e+04	3.1e+03/**1.1e+03**	1.3e+07/**2.9e+05**
countFeatureUses	F	2.8e+08/**1.6e+08**	**7.7e+08**/8.4e+08	6.6e+06/**1.0e+05**	2.8e+03/**7.5e+02**	3.3e+04/**1.3e+04**	2.7e+03/**7.3e+02**	1.5e+07/**2.0e+06**
countLeaves	F	2.8e+08/**1.6e+08**	5.7e+08/**5.5e+08**	5.9e+06/**1.1e+05**	4.3e+03/**1.0e+03**	3.7e+04/**8.7e+03**	4.2e+03/**9.9e+02**	1.4e+07/**3.5e+05**
countNodes	F	2.8e+08/**1.6e+08**	5.7e+08/**5.5e+08**	6.2e+06/**1.1e+05**	5.0e+03/**1.0e+03**	3.9e+04/**8.6e+03**	4.9e+03/**9.9e+02**	1.4e+07/**3.6e+05**
countSmallLeaves	F	2.9e+08/**1.7e+08**	**6.6e+08**/7.8e+08	5.1e+06/**6.2e+04**	4.2e+03/**8.7e+02**	3.4e+04/**1.3e+04**	4.0e+03/**8.5e+02**	1.5e+07/**2.0e+06**
inferenceCost	F	2.8e+08/**1.7e+08**	6.9e+08/**6.9e+08**	8.8e+06/**1.5e+05**	2.9e+03/**6.7e+02**	2.5e+04/**8.0e+03**	2.9e+03/**6.5e+02**	1.4e+07/**3.4e+05**
sumImpurity	F	2.8e+08/**1.6e+08**	**7.0e+08**/7.8e+08	6.9e+06/**3.5e+05**	3.4e+03/**8.5e+02**	3.7e+04/**1.2e+04**	3.3e+03/**8.3e+02**	1.4e+07/**2.0e+06**
sumPathLengths	F	2.9e+08/**1.7e+08**	**7.4e+08**/8.7e+08	7.6e+06/**2.0e+05**	4.3e+03/**7.7e+02**	4.1e+04/**1.3e+04**	4.2e+03/**7.4e+02**	1.5e+07/**2.0e+06**
sumSamples	F	2.8e+08/**1.7e+08**	**5.9e+08**/7.1e+08	6.0e+06/**1.8e+05**	2.8e+03/**1.7e+03**	3.6e+04/**1.5e+04**	2.8e+03/**1.7e+03**	1.5e+07/**2.0e+06**
treeDepth	F	2.8e+08/**1.7e+08**	6.9e+08/**6.9e+08**	8.6e+06/**1.4e+05**	2.3e+03/**7.0e+02**	1.9e+04/**8.0e+03**	2.2e+03/**6.7e+02**	1.4e+07/**3.3e+05**
Total		2.1e+10/1.7e+10	4.3e+10/6.3e+10	5.9e+08/2.9e+07	1.9e+05/1.1e+05	1.9e+06/2.1e+06	1.9e+05/1.0e+05	1.1e+09/3.0e+07

Table 14: Per-pass PAPI counters for DecisionTree (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
classify Batch	F	**2.0e+10**/1.2e+11	**6.6e+10**/2.4e+11	1.2e+08/**4.8e+07**	**3.3e+04**/5.7e+04	**5.0e+05**/2.3e+06	**3.2e+04**/3.7e+04	8.7e+08/**1.8e+07**
classify Depth	F	**2.9e+08**/1.8e+09	**9.4e+08**/3.4e+09	1.6e+06/**6.8e+05**	**5.6e+02**/8.7e+02	**5.1e+03**/3.2e+04	**5.4e+02**/7.0e+02	1.2e+07/**2.9e+05**
classify tree	F	**2.9e+08**/1.8e+09	**9.4e+08**/3.4e+09	1.6e+06/**6.8e+05**	**4.9e+02**/1.5e+03	**4.9e+03**/3.2e+04	**4.7e+02**/7.5e+02	1.2e+07/**2.8e+05**
countFeatureUses	F	**7.2e+08**/2.0e+09	**1.9e+09**/4.0e+09	**2.0e+06**/3.7e+06	**2.8e+02**/2.4e+03	**4.3e+03**/7.4e+04	**2.7e+02**/1.2e+03	9.2e+06/**1.1e+06**
countLeaves	F	**7.3e+08**/1.9e+09	**1.7e+09**/3.8e+09	6.1e+06/**7.3e+05**	**6.5e+02**/3.0e+03	**7.9e+03**/3.4e+04	6.2e+02/**3.7e+02**	9.1e+06/**2.9e+05**
countNodes	F	**7.2e+08**/1.9e+09	**1.7e+09**/3.8e+09	3.2e+06/**7.4e+05**	**1.1e+03**/3.0e+03	**7.3e+03**/3.4e+04	1.0e+03/**5.8e+02**	9.3e+06/**3.1e+05**
countSmallLeaves	F	**7.2e+08**/1.6e+09	**1.8e+09**/3.9e+09	**1.2e+06**/3.6e+06	**9.4e+02**/1.7e+03	**4.5e+03**/7.8e+04	**9.1e+02**/1.0e+03	9.2e+06/**1.2e+06**
inferenceCost	F	**7.3e+08**/1.9e+09	**1.8e+09**/3.8e+09	5.2e+06/**7.3e+05**	**1.1e+03**/4.1e+03	**7.3e+03**/3.5e+04	1.1e+03/**5.3e+02**	9.2e+06/**3.1e+05**
sumImpurity	F	**7.1e+08**/2.0e+09	**1.8e+09**/3.8e+09	**1.0e+06**/3.6e+06	**3.2e+02**/6.0e+02	**4.0e+03**/7.3e+04	**3.1e+02**/5.4e+02	9.4e+06/**1.1e+06**
sumPathLengths	F	**7.4e+08**/1.9e+09	**1.9e+09**/3.9e+09	**1.4e+06**/3.7e+06	**5.5e+02**/9.7e+02	**4.8e+03**/7.9e+04	**5.2e+02**/5.9e+02	9.1e+06/**1.2e+06**
sumSamples	F	**7.1e+08**/1.9e+09	**1.7e+09**/3.8e+09	**9.1e+05**/3.7e+06	**3.5e+02**/8.9e+02	**4.3e+03**/7.9e+04	**3.3e+02**/5.1e+02	9.3e+06/**1.1e+06**
treeDepth	F	**7.3e+08**/1.9e+09	**1.8e+09**/3.8e+09	5.3e+06/**7.2e+05**	**5.7e+02**/3.5e+03	**7.1e+03**/3.4e+04	5.6e+02/**5.1e+02**	9.2e+06/**3.0e+05**
Total		2.7e+10/1.4e+11	8.4e+10/2.8e+11	1.5e+08/7.1e+07	4.0e+04/7.9e+04	5.6e+05/2.8e+06	3.9e+04/4.4e+04	9.7e+08/2.6e+07

Table 15: Per-pass performance for DomTree, ADT has 15 fields, SoA uses 14 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
SumArea	F	6/15	60%	0.0373±0.00001	0.0736±0.00002	**0.0193±0.00002**	0.1959±0.00159	**1.93×**	**1.97×**	**3.82×**	0.38×
computeWidths	M	14/15	7%	**0.0299±0.00004**	0.0314±0.00004	0.0489±0.00004	0.1003±0.00008	0.61×	1.05×	0.64×	0.31×
count styled	F	3/15	80%	0.0366±0.00001	0.0726±0.00084	**0.0118±0.00001**	0.1862±0.00116	**3.11×**	**1.98×**	**6.18×**	0.39×
find max Bottom	F	5/15	67%	0.0363±0.00001	0.0723±0.00002	**0.0203±0.00085**	0.1983±0.00168	**1.78×**	**1.99×**	**3.55×**	0.36×
scaleLayout	M	15/15	0%	**0.0289±0.00003**	0.0306±0.00004	0.0392±0.00004	0.0909±0.00018	0.74×	1.06×	0.78×	0.34×
sumTextWidth	F	3/15	80%	0.0363±0.00001	0.0725±0.00002	**0.0115±0.00002**	0.2556±0.00112	**3.16×**	**2.00×**	**6.31×**	0.28×
Total				0.2053	0.3529	**0.1510**	1.027	**1.36×**	**1.72×**	**2.34×**	0.34×
Geomean								**1.58×**	**1.61×**	**2.53×**	0.34×

Table 16: Per-pass PAPI counters for DomTree (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
SumArea	F	1.9e+08/**9.8e+07**	**2.7e+08**/4.2e+08	2.5e+06/**4.9e+05**	2.3e+03/**2.9e+02**	1.8e+05/**1.2e+04**	2.3e+03/**2.8e+02**	1.3e+07/**2.0e+06**
computeWidths	M	**6.6e+07**/1.6e+08	**1.4e+08**/5.8e+08	**1.3e+06**/1.4e+06	**4.2e+04**/1.4e+05	**5.1e+03**/1.2e+04	**2.6e+03**/1.8e+04	1.1e+06/**4.3e+05**
count styled	F	1.9e+08/**5.9e+07**	**2.5e+08**/2.6e+08	2.1e+06/**2.8e+04**	2.5e+03/**3.2e+02**	2.7e+05/**5.0e+03**	2.4e+03/**3.0e+02**	1.4e+07/**6.5e+05**
find max Bottom	F	1.8e+08/**1.0e+08**	**2.6e+08**/4.7e+08	2.4e+06/**9.7e+05**	6.6e+02/**2.4e+02**	1.6e+05/**9.5e+03**	6.5e+02/**2.2e+02**	1.4e+07/**1.4e+06**
scaleLayout	M	**6.1e+07**/1.1e+08	**8.8e+07**/3.6e+08	**9.9e+05**/1.3e+06	**5.9e+04**/6.6e+04	1.6e+04/**8.2e+03**	**5.3e+03**/1.8e+04	1.3e+06/**4.6e+05**
sumTextWidth	F	1.8e+08/**5.8e+07**	**2.0e+08**/2.4e+08	2.4e+06/**2.1e+04**	2.4e+03/**2.7e+02**	3.9e+05/**4.3e+03**	2.4e+03/**2.6e+02**	1.3e+07/**6.6e+05**
Total		8.7e+08/5.9e+08	1.2e+09/2.3e+09	1.2e+07/4.2e+06	1.1e+05/2.1e+05	1.0e+06/5.1e+04	1.6e+04/3.6e+04	5.6e+07/5.5e+06

Table 17: Per-pass PAPI counters for DomTree (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
SumArea	F	**3.7e+08**/9.9e+08	**6.4e+08**/2.1e+09	8.5e+06/**3.3e+06**	**4.9e+02**/2.2e+03	**2.0e+04**/5.2e+04	**4.7e+02**/2.1e+03	9.9e+06/**1.3e+06**
computeWidths	M	**7.3e+07**/4.2e+08	**1.8e+08**/1.0e+09	**1.3e+06**/3.2e+06	**4.5e+04**/3.1e+05	**4.4e+03**/1.6e+04	**2.5e+03**/2.0e+04	1.1e+06/**4.3e+05**
count styled	F	**3.7e+08**/9.4e+08	**6.3e+08**/2.1e+09	9.9e+06/**1.3e+06**	**3.5e+02**/2.0e+03	**2.3e+04**/3.9e+04	**3.3e+02**/6.0e+02	1.0e+07/**4.3e+05**
find max Bottom	F	**3.6e+08**/1.0e+09	**6.6e+08**/2.2e+09	8.8e+06/**3.0e+06**	**4.7e+02**/2.4e+03	**2.2e+04**/5.6e+04	**4.5e+02**/2.0e+03	1.0e+07/**1.0e+06**
scaleLayout	M	**6.9e+07**/3.7e+08	**1.3e+08**/8.4e+08	**8.9e+05**/2.7e+06	**3.8e+04**/1.7e+05	**3.8e+03**/1.6e+04	**2.7e+03**/2.2e+04	1.1e+06/**4.6e+05**
sumTextWidth	F	**3.7e+08**/1.3e+09	**5.8e+08**/2.1e+09	8.7e+06/**1.3e+06**	**4.1e+02**/3.8e+03	1.2e+05/**4.1e+04**	**3.9e+02**/1.1e+03	1.0e+07/**4.2e+05**
Total		1.6e+09/5.0e+09	2.8e+09/1.0e+10	3.8e+07/1.5e+07	8.5e+04/4.9e+05	2.0e+05/2.2e+05	6.8e+03/4.8e+04	4.2e+07/4.0e+06

Table 18: Per-pass performance for `KDTree`, ADT has 19 fields, SoA uses 12 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
Find nearest Neighbour	F	11/19	42%	0.3742±0.00083	0.4441±0.00211	**0.2939±0.00217**	2.627±0.0042	**1.27×**	**1.19×**	**1.51×**	0.17×
Total				0.3742	0.4441	**0.2939**	2.627	**1.27×**	**1.19×**	**1.51×**	0.17×
Geomean								**1.27×**	**1.19×**	**1.51×**	0.17×

Table 19: Per-pass PAPI counters for `KDTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
Find nearest Neighbour	F	1.9e+09/**1.5e+09**	**6.9e+09**/8.0e+09	2.1e+07/**3.7e+06**	1.9e+03/**9.3e+02**	**3.8e+04**/1.4e+05	1.8e+03/**9.1e+02**	8.2e+07/**2.4e+07**
Total		1.9e+09/1.5e+09	6.9e+09/8.0e+09	2.1e+07/3.7e+06	1.9e+03/9.3e+02	3.8e+04/1.4e+05	1.8e+03/9.1e+02	8.2e+07/2.4e+07

Table 20: Per-pass PAPI counters for `KDTree` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
Find nearest Neighbour	F	**2.3e+09**/1.3e+10	**8.9e+09**/2.2e+10	**1.3e+07**/2.9e+07	**1.1e+03**/3.4e+04	**2.8e+04**/6.1e+05	**1.0e+03**/3.1e+04	8.3e+07/**1.6e+07**
Total		2.3e+09/1.3e+10	8.9e+09/2.2e+10	1.3e+07/2.9e+07	1.1e+03/3.4e+04	2.8e+04/6.1e+05	1.0e+03/3.1e+04	8.3e+07/1.6e+07

Table 21: Per-pass performance for `LinearListReduction`, ADT has 11 fields, SoA uses 11 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
reduction	F	2/11	82%	0.0315±0.00001	0.1630±0.00116	**0.0137±0.00003**	*OOM*	**2.30×**	**5.17×**	**11.89×**	–
Total				0.0315	0.1630	**0.0137**	–	**2.30×**	**5.17×**	**11.89×**	–
Geomean								**2.30×**	**5.17×**	**11.89×**	–

Table 22: Per-pass PAPI counters for `LinearListReduction` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
reduction	F	1.6e+08/**6.9e+07**	**9.0e+07**/1.0e+08	1.7e+06/**1.2e+05**	1.2e+03/**2.6e+02**	2.7e+05/**4.6e+03**	1.2e+03/**2.5e+02**	1.2e+07/**7.3e+05**
Total		1.6e+08/6.9e+07	9.0e+07/1.0e+08	1.7e+06/1.2e+05	1.2e+03/2.6e+02	2.7e+05/4.6e+03	1.2e+03/2.5e+02	1.2e+07/7.3e+05

Table 23: Per-pass PAPI counters for `LinearListReduction` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
reduction	F	8.3e+08/–	3.7e+08/–	5.0e+06/–	2.6e+03/–	3.3e+05/–	2.6e+03/–	3.0e+07/–
Total		8.3e+08/–	3.7e+08/–	5.0e+06/–	2.6e+03/–	3.3e+05/–	2.6e+03/–	3.0e+07/–

Table 24: Per-pass performance for `List`, ADT has 2 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
add1 List	M	2/2	0%	**0.2555±0.00157**	*OOM*	0.3131±0.00166	*OOM*	0.82×	–	–	–
length List	M	1/2	50%	0.0472±0.00004	*OOM*	**0.0263±0.00004**	*OOM*	**1.80×**	–	–	–
sumList	F	2/2	0%	0.1378±0.00003	*OOM*	**0.1321±0.00021**	*OOM*	1.04×	–	–	–
sumList tail recursive	F	2/2	0%	0.1389±0.00112	*OOM*	**0.1324±0.00005**	*OOM*	1.05×	–	–	–
Total				**0.5794**	–	0.6039	–	0.96×	–	–	–
Geomean								**1.13×**	–	–	–

Table 25: Per-pass PAPI counters for `List` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add1 List	M	**6.0e+08**/8.4e+08	**1.9e+09**/3.1e+09	7.8e+06/**1.0e+06**	**3.4e+05**/5.4e+05	**1.4e+04**/5.6e+04	**1.2e+04**/2.2e+04	9.3e+06/**8.1e+06**
length List	M	2.4e+08/**1.3e+08**	8.0e+08/**8.0e+08**	4.0e+06/**2.5e+05**	3.8e+02/**2.7e+02**	**3.5e+03**/1.8e+04	3.6e+02/**2.6e+02**	1.3e+07/**1.1e+06**
sumList	F	7.0e+08/**6.6e+08**	**9.0e+08**/1.1e+09	**8.4e+04**/1.1e+06	**2.9e+02**/5.0e+02	**2.4e+03**/5.1e+04	**2.8e+02**/4.8e+02	8.3e+06/**8.0e+06**
sumList tail recursive	F	7.0e+08/**6.7e+08**	**9.0e+08**/1.1e+09	**9.2e+04**/1.1e+06	4.9e+02/**3.2e+02**	**2.6e+03**/5.1e+04	4.7e+02/**3.0e+02**	8.3e+06/**8.0e+06**
Total		2.2e+09/2.3e+09	4.5e+09/6.1e+09	1.2e+07/3.5e+06	3.4e+05/5.4e+05	2.3e+04/1.8e+05	1.3e+04/2.3e+04	3.9e+07/2.5e+07

Table 26: Per-pass PAPI counters for `List` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
Total		–/–	–/–	–/–	–/–	–/–	–/–	–/–

Table 27: Per-pass performance for `MonoTree`, ADT has 3 fields, SoA uses 2 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
add1Tree	M	3/3	0%	0.0470±0.00028	0.0656±0.00041	**0.0431±0.00038**	0.1305±0.00113	1.09×	**1.40×**	**1.52×**	0.50×
sumTree	F	3/3	0%	0.0115±0.00000	0.0424±0.00085	**0.0114±0.00002**	0.0817±0.00001	1.02×	**3.67×**	**3.73×**	0.52×
sumTree TailRec	F	3/3	0%	0.0120±0.00001	0.0425±0.00001	**0.0111±0.00002**	0.0828±0.00015	1.08×	**3.55×**	**3.81×**	0.51×
Total				0.0705	0.1505	**0.0656**	0.2949	1.08×	**2.13×**	**2.29×**	0.51×
Geomean								1.06×	**2.63×**	**2.79×**	0.51×

Table 28: Per-pass PAPI counters for `MonoTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add1Tree	M	1.7e+08/**1.5e+08**	**5.0e+08**/6.4e+08	**1.8e+05**/3.9e+05	**6.6e+04**/7.8e+04	**2.5e+03**/5.9e+03	4.4e+03/**3.8e+03**	5.0e+05/**4.6e+05**
sumTree	F	5.8e+07/**5.7e+07**	**2.1e+08**/2.5e+08	**4.4e+03**/5.0e+04	**3.6e+02**/1.0e+03	**5.2e+02**/4.4e+03	**3.4e+02**/9.7e+02	7.1e+05/**6.4e+05**
sumTree TailRec	F	6.1e+07/**5.6e+07**	**1.9e+08**/2.2e+08	**4.0e+03**/3.7e+04	**4.1e+02**/8.9e+02	**5.4e+02**/4.2e+03	**3.9e+02**/8.6e+02	6.9e+05/**6.1e+05**
Total		2.9e+08/2.6e+08	9.1e+08/1.1e+09	1.9e+05/4.8e+05	6.7e+04/8.0e+04	3.6e+03/1.5e+04	5.1e+03/5.6e+03	1.9e+06/1.7e+06

Table 29: Per-pass PAPI counters for `MonoTree` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
add1Tree	M	**2.7e+08**/5.9e+08	**9.1e+08**/1.4e+09	**2.3e+05**/1.2e+06	**3.4e+04**/5.9e+04	**1.8e+03**/2.2e+04	2.8e+03/**2.8e+03**	4.6e+05/**3.7e+05**
sumTree	F	**2.1e+08**/4.1e+08	**6.0e+08**/7.6e+08	**8.0e+04**/1.2e+06	**2.8e+02**/4.5e+02	**8.2e+02**/2.0e+04	**2.7e+02**/4.0e+02	3.5e+05/**3.5e+05**
sumTree TailRec	F	**2.2e+08**/4.2e+08	**6.0e+08**/7.6e+08	**9.0e+04**/1.2e+06	**3.3e+02**/3.9e+02	**8.7e+02**/2.0e+04	**3.2e+02**/3.6e+02	3.4e+05/**3.4e+05**
Total		7.0e+08/1.4e+09	2.1e+09/2.9e+09	4.0e+05/3.6e+06	3.5e+04/6.0e+04	3.5e+03/6.3e+04	3.4e+03/3.5e+03	1.2e+06/1.1e+06

Table 30: Per-pass performance for `ObjectGraph`, ADT has 5 fields, SoA uses 4 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
clearMarks	M	4/5	20%	**0.0094±0.00002**	0.0118±0.00002	0.0103±0.00002	0.0263±0.00002	0.92×	**1.25×**	**1.15×**	0.45×
countLargeItems	F	3/5	40%	0.0017±0.00000	0.0055±0.00000	**0.0014±0.00000**	0.0064±0.00000	**1.26×**	**3.22×**	**4.07×**	0.87×
countMarked	F	3/5	40%	0.0017±0.00000	0.0055±0.00000	**0.0014±0.00000**	0.0061±0.00000	**1.26×**	**3.23×**	**4.09×**	0.90×
inflateSizes	M	5/5	0%	**0.0091±0.00001**	0.0116±0.00001	0.0099±0.00066	0.0267±0.00001	0.92×	**1.27×**	**1.17×**	0.43×
sumObjIds	F	3/5	40%	0.0017±0.00000	0.0054±0.00000	**0.0013±0.00000**	0.0062±0.00000	**1.28×**	**3.26×**	**4.16×**	0.87×
totalHeapSize	F	3/5	40%	0.0017±0.00027	0.0054±0.00052	**0.0013±0.00020**	0.0062±0.00037	**1.26×**	**3.23×**	**4.08×**	0.87×
Total				**0.0253**	0.0452	0.0255	0.0780	0.99×	**1.79×**	**1.77×**	0.58×
Geomean								**1.14×**	**2.36×**	**2.69×**	0.70×

Table 31: Per-pass PAPI counters for `ObjectGraph` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
clearMarks	M	**2.5e+07**/2.9e+07	**6.1e+07**/1.1e+08	1.1e+05/**3.6e+04**	2.0e+04/**1.4e+04**	**1.0e+03**/1.1e+03	1.2e+03/**9.5e+02**	1.2e+05/**5.2e+04**
countLargeItems	F	8.7e+06/**6.9e+06**	**3.5e+07**/3.9e+07	5.6e+04/**8.8e+03**	3.0e+02/**2.9e+02**	**3.4e+02**/5.9e+02	2.9e+02/**2.7e+02**	8.5e+04/**1.8e+03**
countMarked	F	8.7e+06/**6.9e+06**	**3.5e+07**/3.9e+07	5.6e+04/**8.6e+03**	3.2e+02/**2.9e+02**	**3.3e+02**/6.6e+02	3.0e+02/**2.8e+02**	8.4e+04/**1.0e+03**
inflateSizes	M	**2.4e+07**/2.8e+07	**6.6e+07**/1.2e+08	1.3e+05/**4.8e+04**	**1.6e+04**/1.9e+04	**8.6e+02**/1.4e+03	1.6e+03/**1.1e+03**	1.2e+05/**5.9e+04**
sumObjIds	F	8.4e+06/**6.6e+06**	**3.1e+07**/3.6e+07	7.6e+04/**5.4e+03**	3.2e+02/**2.7e+02**	**3.9e+02**/5.5e+02	3.1e+02/**2.6e+02**	8.1e+04/**7.1e+02**
totalHeapSize	F	8.4e+06/**6.6e+06**	**3.1e+07**/3.6e+07	7.8e+04/**6.9e+03**	3.3e+02/**2.7e+02**	**3.8e+02**/6.0e+02	3.1e+02/**2.6e+02**	8.5e+04/**2.5e+03**
Total		8.4e+07/8.5e+07	2.6e+08/3.8e+08	5.0e+05/1.1e+05	3.8e+04/3.5e+04	3.3e+03/5.0e+03	4.0e+03/3.2e+03	5.7e+05/1.2e+05

Table 32: Per-pass PAPI counters for `ObjectGraph` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
clearMarks	M	**3.7e+07**/1.1e+08	**1.1e+08**/2.9e+08	**4.0e+04**/2.8e+05	**1.1e+04**/2.0e+04	**6.4e+02**/3.5e+03	**7.8e+02**/8.0e+02	1.1e+05/**6.7e+04**
countLargeItems	F	**2.8e+07**/3.2e+07	**8.2e+07**/1.4e+08	**1.6e+04**/1.3e+05	**2.4e+02**/2.5e+02	**3.2e+02**/1.8e+03	**2.3e+02**/2.4e+02	8.6e+04/**1.8e+03**
countMarked	F	**2.8e+07**/3.1e+07	**8.2e+07**/1.4e+08	**1.6e+04**/1.3e+05	**2.2e+02**/2.3e+02	**3.2e+02**/1.4e+03	**2.1e+02**/2.2e+02	8.4e+04/**7.2e+02**
inflateSizes	M	**3.7e+07**/1.1e+08	**1.2e+08**/3.0e+08	**2.4e+04**/3.3e+05	**1.3e+04**/1.9e+04	**5.5e+02**/3.2e+03	**8.0e+02**/1.1e+03	1.1e+05/**6.6e+04**
sumObjIds	F	**2.7e+07**/3.2e+07	**8.0e+07**/1.4e+08	**1.6e+04**/1.4e+05	**2.3e+02**/2.3e+02	**3.1e+02**/2.3e+03	**2.2e+02**/2.3e+02	8.4e+04/**7.8e+02**
totalHeapSize	F	**2.7e+07**/3.1e+07	**8.0e+07**/1.4e+08	**1.6e+04**/1.3e+05	**2.4e+02**/2.9e+02	**3.1e+02**/1.8e+03	**2.3e+02**/2.8e+02	8.6e+04/**2.4e+03**
Total		1.8e+08/3.5e+08	5.6e+08/1.1e+09	1.3e+05/1.1e+06	2.5e+04/3.9e+04	2.5e+03/1.4e+04	2.5e+03/2.9e+03	5.6e+05/1.4e+05

Table 33: Per-pass performance for `OctTree`, ADT has 16 fields, SoA uses 9 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
clearFlags	M	15/16	6%	**0.0191±0.00001**	0.0193±0.00002	0.0244±0.00002	0.0685±0.00002	0.78×	1.01×	0.79×	0.28×
countActive	F	9/16	44%	0.0039±0.00000	0.0056±0.00001	**0.0033±0.00000**	0.0278±0.00000	**1.18×**	**1.46×**	**1.71×**	0.20×
countParticles	F	8/16	50%	0.0039±0.00000	0.0054±0.00000	**0.0030±0.00000**	0.0264±0.00001	**1.28×**	**1.38×**	**1.78×**	0.20×
scaleEnergy	M	16/16	0%	**0.0193±0.00004**	0.0195±0.00004	0.0248±0.00004	0.0709±0.00005	0.78×	1.01×	0.79×	0.28×
sumEnergy	F	10/16	38%	0.0039±0.00000	0.0056±0.00006	**0.0034±0.00000**	0.0282±0.00012	**1.15×**	**1.43×**	**1.64×**	0.20×
sumMass	F	9/16	44%	0.0039±0.00001	0.0056±0.00038	**0.0034±0.00029**	0.0278±0.00028	**1.16×**	**1.42×**	**1.65×**	0.20×
Total				**0.0541**	0.0610	0.0624	0.2496	0.87×	**1.13×**	0.98×	0.24×
Geomean								1.03×	**1.27×**	**1.31×**	0.22×

Table 34: Per-pass PAPI counters for `OctTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
clearFlags	M	**4.6e+07**/6.7e+07	**1.0e+08**/2.6e+08	**7.6e+04**/4.6e+05	**2.4e+04**/8.0e+04	**1.4e+03**/5.2e+03	**1.2e+03**/2.3e+03	5.7e+05/**2.4e+05**
countActive	F	2.0e+07/**1.7e+07**	5.1e+07/**4.9e+07**	4.9e+04/**4.7e+04**	3.3e+02/**2.4e+02**	**5.0e+02**/1.6e+03	3.2e+02/**2.3e+02**	6.0e+05/**2.2e+03**
countParticles	F	2.0e+07/**1.5e+07**	5.2e+07/**5.2e+07**	5.9e+04/**2.9e+04**	3.5e+02/**2.4e+02**	**6.3e+02**/8.8e+02	3.4e+02/**2.2e+02**	6.0e+05/**2.9e+02**
scaleEnergy	M	**4.6e+07**/6.9e+07	**1.0e+08**/2.7e+08	**7.3e+04**/4.7e+05	**2.5e+04**/5.9e+04	**1.4e+03**/5.8e+03	**1.1e+03**/2.5e+03	5.7e+05/**2.5e+05**
sumEnergy	F	2.0e+07/**1.7e+07**	**5.2e+07**/6.1e+07	2.5e+04/**2.1e+04**	3.0e+02/**2.4e+02**	**4.3e+02**/1.5e+03	2.8e+02/**2.3e+02**	6.0e+05/**4.8e+04**
sumMass	F	2.0e+07/**1.7e+07**	**5.2e+07**/6.1e+07	**2.4e+04**/2.8e+04	2.9e+02/**2.6e+02**	**4.2e+02**/1.6e+03	2.7e+02/**2.4e+02**	6.1e+05/**4.8e+04**
Total		1.7e+08/2.0e+08	4.1e+08/7.5e+08	3.1e+05/1.1e+06	5.0e+04/1.4e+05	4.7e+03/1.7e+04	3.4e+03/5.7e+03	3.5e+06/6.0e+05

Table 35: Per-pass PAPI counters for `OctTree` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
clearFlags	M	**4.6e+07**/2.9e+08	**1.3e+08**/6.5e+08	**1.5e+05**/1.0e+06	**2.6e+04**/1.5e+05	**1.2e+03**/9.2e+03	**1.4e+03**/3.1e+03	5.6e+05/**2.4e+05**
countActive	F	**2.8e+07**/1.4e+08	**8.9e+07**/2.5e+08	1.5e+05/**7.8e+04**	**2.7e+02**/5.3e+02	**6.0e+02**/4.4e+03	**2.6e+02**/2.7e+02	5.2e+05/**6.7e+03**
countParticles	F	**2.7e+07**/1.3e+08	**8.3e+07**/2.5e+08	4.7e+04/**4.2e+04**	**3.0e+02**/6.1e+02	**6.2e+02**/4.1e+03	2.8e+02/**2.5e+02**	5.5e+05/**3.2e+03**
scaleEnergy	M	**4.6e+07**/3.0e+08	**1.4e+08**/6.5e+08	**1.4e+05**/1.1e+06	**2.7e+04**/1.4e+05	**1.2e+03**/8.1e+03	**1.4e+03**/5.1e+03	5.5e+05/**2.5e+05**
sumEnergy	F	**2.8e+07**/1.4e+08	**9.1e+07**/2.5e+08	**1.3e+04**/3.3e+05	**2.7e+02**/6.2e+02	**5.5e+02**/8.1e+03	**2.6e+02**/3.5e+02	5.4e+05/**5.8e+04**
sumMass	F	**2.8e+07**/1.4e+08	**9.1e+07**/2.5e+08	**1.4e+04**/3.3e+05	**2.4e+02**/5.0e+02	**5.7e+02**/8.3e+03	**2.3e+02**/3.6e+02	5.5e+05/**5.6e+04**
Total		2.0e+08/1.2e+09	6.2e+08/2.3e+09	5.2e+05/2.9e+06	5.4e+04/3.0e+05	4.8e+03/4.2e+04	3.8e+03/9.4e+03	3.3e+06/6.1e+05

Table 36: Per-pass performance for `PiecewiseFunctions`, ADT has 8 fields, SoA uses 7 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
add constant	M	8/8	0%	**0.0148±0.00002**	0.0168±0.00001	0.0168±0.00002	0.0594±0.00002	0.88×	**1.14×**	1.00×	0.28×
countSplit	F	3/8	62%	0.0026±0.00001	0.0068±0.00000	**0.0013±0.00000**	0.0338±0.00001	**1.95×**	**2.63×**	**5.13×**	0.20×
differentiate	M	8/8	0%	**0.0149±0.00002**	0.0171±0.00001	0.0169±0.00012	0.0559±0.00002	0.88×	**1.15×**	1.02×	0.31×
max degree	F	3/8	62%	0.0028±0.00001	0.0059±0.00000	**0.0015±0.00000**	0.0273±0.00001	**1.92×**	**2.09×**	**4.01×**	0.22×
square	M	8/8	0%	**0.0153±0.00062**	0.0170±0.00003	0.0171±0.00004	0.0592±0.00097	0.89×	**1.12×**	1.00×	0.29×
sum co-efficients	F	3/8	62%	0.0026±0.00018	0.0060±0.00047	**0.0013±0.00020**	0.0335±0.00135	**1.98×**	**2.31×**	**4.56×**	0.18×
sumError	F	3/8	62%	0.0025±0.00001	0.0059±0.00000	**0.0013±0.00001**	0.0316±0.00001	**1.96×**	**2.36×**	**4.63×**	0.19×
Total				**0.0555**	0.0757	0.0562	0.3007	0.99×	**1.36×**	**1.35×**	0.25×
Geomean								**1.39×**	**1.72×**	**2.39×**	0.23×

Table 37: Per-pass PAPI counters for `PiecewiseFunctions` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add constant	M	**3.4e+07**/4.5e+07	**7.4e+07**/1.7e+08	**1.5e+05**/2.7e+05	**2.1e+04**/2.5e+04	**1.5e+03**/3.1e+03	**1.5e+03**/2.7e+03	4.4e+05/**1.8e+05**
countSplit	F	1.3e+07/**6.8e+06**	**3.5e+07**/3.9e+07	3.9e+05/**1.5e+04**	5.2e+02/**2.8e+02**	2.8e+03/**6.7e+02**	5.1e+02/**2.7e+02**	4.7e+05/**9.0e+02**
differentiate	M	**3.5e+07**/4.5e+07	**7.5e+07**/1.8e+08	**1.2e+05**/2.6e+05	3.6e+04/**2.7e+04**	**1.7e+03**/3.0e+03	**2.3e+03**/3.6e+03	4.7e+05/**1.8e+05**
max degree	F	1.4e+07/**7.5e+06**	**3.4e+07**/3.7e+07	2.9e+05/**5.3e+03**	5.0e+02/**2.7e+02**	1.4e+03/**7.2e+02**	4.7e+02/**2.6e+02**	4.8e+05/**5.5e+02**
square	M	**3.5e+07**/4.5e+07	**7.4e+07**/1.7e+08	**1.4e+05**/2.7e+05	3.1e+04/**2.4e+04**	**1.7e+03**/3.1e+03	**2.5e+03**/4.0e+03	4.3e+05/**1.8e+05**
sum co-efficients	F	1.3e+07/**6.4e+06**	**2.8e+07**/3.3e+07	3.9e+05/**7.8e+03**	5.0e+02/**3.1e+02**	3.1e+03/**7.6e+02**	4.8e+02/**2.9e+02**	5.1e+05/**3.7e+02**
sumError	F	1.3e+07/**6.5e+06**	**2.8e+07**/3.3e+07	3.9e+05/**8.4e+03**	4.8e+02/**2.8e+02**	3.4e+03/**7.1e+02**	4.7e+02/**2.6e+02**	4.8e+05/**6.2e+02**
Total		1.6e+08/1.6e+08	3.5e+08/6.6e+08	1.9e+06/8.4e+05	9.0e+04/7.8e+04	1.6e+04/1.2e+04	8.3e+03/1.1e+04	3.3e+06/5.4e+05

Table 38: Per-pass PAPI counters for `PiecewiseFunctions` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
add constant	M	**4.4e+07**/2.6e+08	**1.2e+08**/4.4e+08	**4.4e+04**/6.4e+05	**1.9e+04**/3.4e+04	**7.4e+02**/5.8e+03	**1.3e+03**/5.3e+03	4.0e+05/**1.9e+05**
countSplit	F	**3.4e+07**/1.7e+08	**8.2e+07**/2.0e+08	6.0e+05/**1.5e+05**	**2.7e+02**/5.1e+02	**4.1e+02**/3.4e+03	2.6e+02/**2.4e+02**	3.2e+05/**2.1e+03**
differentiate	M	**4.5e+07**/2.4e+08	**1.3e+08**/4.6e+08	**3.8e+04**/7.2e+05	**2.3e+04**/4.4e+04	**7.2e+02**/7.0e+03	**1.6e+03**/4.3e+03	4.2e+05/**1.9e+05**
max degree	F	**3.0e+07**/1.4e+08	**7.8e+07**/1.9e+08	**2.7e+04**/1.6e+05	**2.5e+02**/4.4e+02	**3.1e+02**/3.7e+03	**2.4e+02**/2.7e+02	3.7e+05/**2.9e+03**
square	M	**4.4e+07**/2.6e+08	**1.2e+08**/4.4e+08	**4.2e+04**/6.2e+05	**1.8e+04**/4.2e+04	**8.0e+02**/6.1e+03	**1.8e+03**/6.0e+03	4.0e+05/**1.9e+05**
sum co-efficients	F	**3.0e+07**/1.7e+08	**7.6e+07**/1.9e+08	**2.7e+04**/1.6e+05	**2.6e+02**/3.9e+02	**3.2e+02**/3.8e+03	**2.4e+02**/2.7e+02	3.8e+05/**3.5e+03**
sumError	F	**3.0e+07**/1.6e+08	**7.6e+07**/1.9e+08	**2.5e+04**/1.6e+05	**2.4e+02**/4.8e+02	**3.2e+02**/3.6e+03	**2.3e+02**/2.7e+02	3.5e+05/**2.9e+03**
Total		2.6e+08/1.4e+09	6.8e+08/2.1e+09	8.0e+05/2.6e+06	6.1e+04/1.2e+05	3.6e+03/3.3e+04	5.7e+03/1.7e+04	2.6e+06/5.8e+05

Table 39: Per-pass performance for `TernaryTree`, ADT has 5 fields, SoA uses 3 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
add 1 tree	M	5/5	0%	0.0840±0.00010	0.0979±0.00020	**0.0786±0.00020**	0.3653±0.00114	1.07×	**1.17×**	**1.25×**	0.27×
sum tree	F	5/5	0%	**0.0304±0.00096**	0.0522±0.00001	0.0313±0.00005	0.2514±0.00059	0.97×	**1.72×**	**1.67×**	0.21×
Total				0.1144	0.1502	**0.1099**	0.6167	1.04×	**1.31×**	**1.37×**	0.24×
Geomean								1.02×	**1.42×**	**1.44×**	0.24×

Table 40: Per-pass PAPI counters for `TernaryTree` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (A/S)	INS (A/S)	L1D (A/S)	L1I (A/S)	L2D (A/S)	L2I (A/S)	LLC (A/S)
add 1 tree	M	2.7e+08/**2.3e+08**	**7.5e+08**/1.1e+09	**2.8e+05**/1.2e+06	**7.4e+04**/1.5e+05	**4.3e+03**/1.1e+04	**3.7e+03**/4.0e+03	1.6e+06/**1.1e+06**
sum tree	F	**1.5e+08**/1.6e+08	**4.8e+08**/5.1e+08	**6.0e+04**/1.2e+05	**2.8e+02**/3.7e+02	**7.7e+02**/8.6e+03	**2.6e+02**/3.5e+02	1.7e+06/**1.2e+06**
Total		4.2e+08/3.9e+08	1.2e+09/1.6e+09	3.4e+05/1.4e+06	7.4e+04/1.5e+05	5.1e+03/2.0e+04	4.0e+03/4.3e+03	3.3e+06/2.3e+06

Table 41: Per-pass PAPI counters for `TernaryTree` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	T	CYC (Ai/Si)	INS (Ai/Si)	L1D (Ai/Si)	L1I (Ai/Si)	L2D (Ai/Si)	L2I (Ai/Si)	LLC (Ai/Si)
add 1 tree	M	**3.4e+08**/1.7e+09	**1.2e+09**/2.7e+09	**2.1e+05**/2.5e+06	**6.9e+04**/1.9e+05	**3.2e+03**/4.5e+04	**4.3e+03**/6.3e+03	1.4e+06/**9.5e+05**
sum tree	F	**2.7e+08**/1.3e+09	**8.3e+08**/1.3e+09	**1.2e+05**/2.7e+06	**2.5e+02**/2.3e+03	**3.1e+03**/4.8e+04	**2.3e+02**/9.5e+02	1.3e+06/**8.6e+05**
Total		6.1e+08/3.0e+09	2.0e+09/4.0e+09	3.3e+05/5.2e+06	6.9e+04/1.9e+05	6.2e+03/9.3e+04	4.6e+03/7.3e+03	2.7e+06/1.8e+06

Table 42: Per-pass performance for `Trie`, ADT has 10 fields, SoA uses 9 buffers (mutable + immutable cursors). Times are median per iteration (s); ± shows standard error of the mean across –iterate runs. T: F=fold, M=map. Uses: fields accessed / total (recursive + non-recursive). Dead%: fraction of fields not accessed by this pass. Speedup >1× means SoA is faster. OOM = out of memory.

Pass	T	Uses	Dead%	Am	Ai	Sm	Si	Am/Sm	Ai/Am	Ai/Sm	Ai/Si
clear trie flags	M	9/10	10%	**0.0376±0.00005**	0.0418±0.00004	0.0465±0.00005	0.1261±0.00005	0.81×	**1.11×**	0.90×	0.33×
count trie flags	F	3/10	70%	0.0066±0.00000	0.0122±0.00000	**0.0029±0.00000**	0.0506±0.00001	**2.30×**	**1.85×**	**4.25×**	0.24×
scale frequency	M	10/10	0%	**0.0375±0.00008**	0.0416±0.00113	0.0480±0.00108	0.1295±0.00007	0.78×	**1.11×**	0.87×	0.32×
sum count words	F	2/10	80%	0.0063±0.00000	0.0133±0.00000	**0.0027±0.00000**	0.0584±0.00081	**2.36×**	**2.11×**	**4.97×**	0.23×
sum frequency	F	3/10	70%	0.0065±0.00009	0.0121±0.00045	**0.0028±0.00019**	0.0553±0.00008	**2.32×**	**1.85×**	**4.29×**	0.22×
sum subtrees	F	3/10	70%	0.0065±0.00001	0.0120±0.00001	**0.0028±0.00000**	0.0553±0.00001	**2.29×**	**1.86×**	**4.25×**	0.22×
Total				**0.1010**	0.1330	0.1057	0.4751	0.96×	**1.32×**	**1.26×**	0.28×
Geomean								**1.62×**	**1.60×**	**2.59×**	0.26×

Table 43: Per-pass PAPI counters for `Trie` (A/S). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (A/S)**	**INS (A/S)**	**L1D (A/S)**	**L1I (A/S)**	**L2D (A/S)**	**L2I (A/S)**	**LLC (A/S)**
clear trie flags	M	**8.3e+07**/1.3e+08	**1.4e+08**/4.3e+08	**3.9e+05**/5.4e+05	**5.0e+04**/6.9e+04	**4.1e+03**/9.1e+03	**1.6e+03**/1.5e+04	1.5e+06/**5.0e+05**
count trie flags	F	3.3e+07/**1.4e+07**	**6.3e+07**/6.4e+07	7.3e+05/**6.0e+03**	2.5e+02/**2.5e+02**	8.3e+03/**1.3e+03**	**2.3e+02**/2.4e+02	1.9e+06/**3.5e+04**
scale frequency	M	**8.2e+07**/1.3e+08	**1.5e+08**/4.5e+08	**4.3e+05**/7.0e+05	**4.8e+04**/9.4e+04	**4.2e+03**/1.1e+04	**1.9e+03**/1.7e+04	1.5e+06/**5.8e+05**
sum count words	F	3.2e+07/**1.3e+07**	**5.1e+07**/5.3e+07	8.2e+05/**2.1e+04**	**2.5e+02**/2.6e+02	2.1e+04/**6.0e+02**	**2.3e+02**/2.5e+02	1.8e+06/**8.6e+02**
sum frequency	F	3.3e+07/**1.4e+07**	**5.7e+07**/5.8e+07	7.9e+05/**6.0e+03**	**2.2e+02**/2.5e+02	9.3e+03/**1.4e+03**	**2.0e+02**/2.3e+02	1.8e+06/**4.0e+04**
sum subtrees	F	3.3e+07/**1.4e+07**	**5.7e+07**/5.8e+07	8.0e+05/**6.1e+03**	2.7e+02/**2.4e+02**	1.7e+04/**1.4e+03**	2.5e+02/**2.4e+02**	1.8e+06/**3.3e+04**
Total		3.0e+08/3.2e+08	5.2e+08/1.1e+09	4.0e+06/1.3e+06	9.8e+04/1.6e+05	6.4e+04/2.5e+04	4.4e+03/3.2e+04	1.0e+07/1.2e+06

Table 44: Per-pass PAPI counters for `Trie` (Ai/Si). Each cell is median counter value pair per pass. Counter headers are abbreviated for compactness: CYC=CPU cycles, L1D/L1I/L2D/L2I=cache misses, LLC=LLC load misses.

Pass	**T**	**CYC (Ai/Si)**	**INS (Ai/Si)**	**L1D (Ai/Si)**	**L1I (Ai/Si)**	**L2D (Ai/Si)**	**L2I (Ai/Si)**	**LLC (Ai/Si)**
clear trie flags	M	**1.0e+08**/5.3e+08	**2.3e+08**/1.1e+09	**1.5e+05**/2.0e+06	**5.1e+04**/1.1e+05	**2.4e+03**/1.4e+04	**2.5e+03**/2.3e+04	1.4e+06/**5.0e+05**
count trie flags	F	**6.2e+07**/2.6e+08	**1.6e+08**/4.5e+08	**7.6e+04**/3.0e+05	**2.3e+02**/6.8e+02	**6.8e+02**/7.9e+03	**2.2e+02**/3.0e+02	1.4e+06/**5.2e+04**
scale frequency	M	**1.0e+08**/5.5e+08	**2.4e+08**/1.1e+09	**1.2e+05**/2.4e+06	**6.6e+04**/1.9e+05	**2.3e+03**/1.7e+04	**2.5e+03**/2.6e+04	1.4e+06/**5.7e+05**
sum count words	F	**6.8e+07**/3.0e+08	**1.4e+08**/4.2e+08	1.1e+06/**6.5e+04**	**2.4e+02**/6.4e+02	**1.3e+03**/4.3e+03	**2.3e+02**/2.5e+02	1.3e+06/**4.4e+03**
sum frequency	F	**6.1e+07**/2.8e+08	**1.5e+08**/4.2e+08	**9.5e+04**/3.1e+05	**2.7e+02**/6.5e+02	**7.1e+02**/8.0e+03	**2.7e+02**/2.9e+02	1.5e+06/**6.5e+04**
sum subtrees	F	**6.1e+07**/2.8e+08	**1.5e+08**/4.2e+08	**9.3e+04**/3.0e+05	**2.3e+02**/6.4e+02	**7.1e+02**/7.9e+03	**2.2e+02**/3.0e+02	1.4e+06/**5.7e+04**
Total		4.6e+08/2.2e+09	1.1e+09/4.0e+09	1.7e+06/5.4e+06	1.2e+05/3.0e+05	8.1e+03/5.8e+04	5.9e+03/5.1e+04	8.5e+06/1.2e+06